

The Legitimate Path

Why Doing It Right Is the Only Way to Get It Done

Abstract

Software engineers working in complex, constrained organisations frequently reach a point of crisis. The structural problems are visible. The correct solutions are understood. The organisation is not acting. In that moment, the temptation is to stop waiting and simply fix it — to route around governance, build the thing that needs building, and present the result as a *fait accompli*.

This paper argues that this move, however emotionally satisfying, is structurally self-defeating. Guerrilla development is not a methodology. It is a hostage situation: a solution that cannot be governed, cannot be migrated from, and cannot be owned by an organisation that never sanctioned it. Worse, by suppressing the visible evidence of structural failure, it perpetuates the conditions that made the failure inevitable — until those conditions reassert themselves, as they inevitably do.

The alternative is harder. It is grinding, exhausting, and offers no guarantee of immediate relief. It requires holding a correct position under sustained pressure, identifying where authority actually sits rather than where it is formally declared, and treating apparent obstacles — risk teams, compliance functions, gatekeepers of every kind — not as adversaries to be bypassed but as stakeholders with legitimate concerns that a well-designed system must satisfy. These are not sequential steps. They are concurrent disciplines, continuously revised as the situation develops.

The paper draws on direct experience of two contrasting conditions: an empowered greenfield delivery operating under minimal organisational impedance, where structural integrity was maintained through rapid change and external disruption; and a scaling organisation carrying significant inherited structural debt, where the same disciplines are being applied under sustained resistance and commercial pressure.

The thesis is simple, and I mean it without qualification: if you do this right, the right thing will happen.

1. The Moment of Temptation

You have been in this situation. The diagnosis is not in dispute — even the people blocking your proposal would probably agree with it in the abstract. The solution is clear, proportionate, and achievable. The roadmap is drafted. The case has been made, more than once, through the appropriate channels.

And yet nothing happens. Meetings occur. Apparent agreement is reached. Resources are promised and then redirected. The problem you identified six months ago is now visibly worse. New people arrive and discover the same problem independently. It gets escalated, discussed, and deferred again.

The temptation at this point is entirely rational. You have the skills. You understand the domain. You could simply start building the thing the organisation needs, demonstrate its value, and present it as a solution rather than a proposal. The *fait accompli* is a real strategy, and engineers who have deployed it will tell you it sometimes works.

I am not here to tell you that the frustration is misplaced. It is not. The frustration is a legitimate response to a real structural condition: an

organisation that can see its own failure and cannot act on it. If you are grinding against that condition and feeling the weight of it, that feeling is accurate. The situation is genuinely difficult, and anyone who tells you otherwise has not spent enough time inside it.

What I want to argue is that the guerrilla move, despite being emotionally available and occasionally locally effective, is the wrong response to that condition — not for ethical reasons, but for structural ones.

2. Guerrilla Development Is a Hostage Situation

The engineer who routes around governance and builds an unsanctioned solution believes they are taking control of the situation. This is an illusion. What they have actually done is created a dependency that now controls them.

The organisation cannot migrate away from the solution without the engineer's involvement. It cannot govern what it does not understand and didn't commission. It cannot audit decisions that were never made explicitly. The engineer is now permanently load-bearing within a structure they built outside the system of record — and the organisation, rather than owning its architecture, now has a dependency on an individual it cannot replace.

This is the hostage situation. Both parties are constrained by it. The engineer cannot leave without abandoning something they care about. The organisation cannot change direction without losing something it depends on but never properly understood. The dysfunction that produced the original problem has not been resolved. It has been buried under a layer of

undocumented, unsanctioned complexity that makes the next failure harder to diagnose and more expensive to fix.

There is a deeper problem. Failure, in a well-functioning system, is information. It is the mechanism by which a system exposes its real architecture, reveals its implicit assumptions, and creates the conditions under which it can be corrected. When failure is suppressed — by a workaround, a patch, an unsanctioned parallel implementation — the organisation loses access to that signal. The structural condition that produced the failure remains intact, invisible, and accumulating pressure.

Guerrilla development is a form of failure suppression. It solves the immediate presenting problem while leaving the underlying cause untouched. The problem will return. When it does, there will be no institutional memory of why the previous workaround was built, what it was trying to address, or where its edges are. The organisation will encounter the same failure as if for the first time, but with additional complexity in its way.

This is not a methodology. It is a way of deferring a reckoning while making it worse.

3. What Coherence Actually Produces

Before describing the legitimate path in detail, it is worth establishing concretely what it makes possible. The argument so far has been defensive: guerrilla development is self-defeating. The affirmative case is equally important.

Consider a greenfield project: a small team, fully empowered, with complete authority over its own delivery. Two engineers, pair-programming throughout. A projected go-live date pushed forward, not back, in response to a major external disruption arriving two months into the project. A working system delivered, then refactored three times in succession while continuing to deliver new functionality at each stage.

That account is not exceptional in terms of individual talent. It is exceptional in terms of the conditions it describes. Two engineers can deliver more, faster, with greater structural integrity, than organisations with ten times the headcount routinely manage. The reason is not skill. The reason is the absence of organisational impedance.

When authority, responsibility, and capability are aligned — when the people doing the work have complete ownership of the decisions that govern it — coherence becomes a force multiplier. Communication overhead drops to near zero. Decisions are made at the point of maximum information. Structural integrity is maintained not through governance processes but through the continuous shared understanding of a small, empowered team.

The ability to say yes to that external disruption — to pivot the product and move the delivery date forward — was not a consequence of individual heroism. It was a consequence of an architecture built, from the beginning, with change as a first-class property. The system could absorb the change without structural damage because it had never been built to resist change.

This is what the legitimate path, under optimal conditions, actually produces. The constrained case — inherited debt, organisational

misalignment, commercial pressure — is harder. But the same disciplines apply, and the same logic holds: coherence, wherever it can be established and maintained, is the condition under which good software becomes possible.

4. The Concurrent Disciplines

There is no five-step methodology for the legitimate path. What follows is not a process to be executed in sequence. It is a set of concurrent disciplines — orientations that operate simultaneously, each informing the others, all subject to continuous revision as the situation develops. The moment you believe you have finished applying any of them, you are probably behind the situation.

4.1 Holding the Position

The first discipline is the simplest to describe and the hardest to sustain: maintaining a correct diagnosis, clearly and formally articulated, in the face of an organisation that is not yet acting on it.

This means describing engineering proposals and options as clearly as possible, utilising appropriate channels to escalate, and backing every claim with as much evidence as can be assembled. It means not going behind anyone's back. It means not improvising solutions outside the system of record. It means not proposing reckless interventions because the pressure to do something has become unbearable.

This is a conscious decision to operate within legitimate channels even when the organisation's incentive structures are working against the correct outcome. It accepts a degree of friction — and possibly a slower path to

resolution, or even the risk of failure — in order to preserve the integrity of the process and the engineer's professional credibility within it.

The cost of that choice is real. The legitimate path offers no emotional relief. You are choosing the harder experience — the grinding, the repeated arguments, the apparent resolutions that produce no substantive response — in service of a structural argument that may not succeed immediately.

But maintaining legibility — keeping the diagnosis visible, the proposals formal, the escalation documented — is itself a form of structural integrity. The paper trail is part of the system. The organisation's ability to govern itself over time depends on decisions and their rationale being traceable. An engineer who routes around that to get a result is, in a precise sense, degrading the system's capacity for self-correction, regardless of whether the immediate result is correct.

4.2 Reading Authority Correctly

Formal authority and actual power frequently diverge. An organisation's chart tells you who is accountable. It does not always tell you who can act.

One of the most demoralising experiences in the legitimate path is apparent resolution that produces no substantive response: a meeting at which agreement seems to have been reached, followed by nothing. This experience has a specific diagnostic value. It is the organisation telling you something about the person you've been talking to — namely, that they have the power to agree but not the power to act.

Identifying where authority actually sits is an exploratory process. There is no shortcut. It requires observing who is in the room when decisions are

actually made, who controls resource allocation in practice rather than on paper, and who bears the consequences when things go wrong. These are not always the same people. In organisations with complex governance, they are often different people operating in different registers.

The discipline is not to become cynical about this, but to become precise. The question is not “who is obstructing me?” but “where does the authority to act actually reside, and what does the person who holds it need to hear?” These are different questions, and they lead to different conversations.

4.3 Converting Obstacles into Stakeholders

The typical engineer reads security teams, risk functions, and compliance gatekeepers as obstacles — people whose job is to say no, whose concerns are orthogonal to delivery, and who therefore need to be minimised, routed around, or endured. The guerrilla instinct is strongest in relation to these teams, because they seem to represent the organisation’s irrationality in its purest form.

This reading is incorrect, and acting on it is costly.

Risk and compliance teams want confidence, reliability, and auditability. These are architectural properties — the same properties that good engineering produces. The divergence between these teams and delivery is not a divergence of goals. It is a divergence of language, timescale, and organisational position. They are expressing the same requirements in a different register.

The move is to treat them as stakeholders with a known form: to model their concerns explicitly, understand what a satisfactory solution looks like from

their perspective, and design for those requirements rather than around them. This is not a political concession. It is better engineering. A system that satisfies auditability, confidence, and reliability as first-class requirements is a better system than one that treats them as constraints to be minimised.

The practical consequence is that teams who began as apparent adversaries become allies. Not because they were persuaded to lower their standards, but because a proposal was put before them that genuinely met those standards. The argument stops being about whether the concerns are legitimate — they are — and starts being about how to satisfy them together.

4.4 Working With the Resource Logic

Organisations allocate resources according to their own internal logic, and that logic is rarely perfectly aligned with engineering necessity. The naive response is to argue that the resource logic is wrong and should be changed. This is sometimes correct, but it is rarely sufficient as a strategy.

The more effective discipline is to work within the existing resource logic while reframing what it is being applied to. If budget has been allocated to delivery teams to solve problems that actually lie in the platform those teams depend on, the argument is not “we need additional resource for platform investment.” The argument is “the resource already committed is being applied to the wrong problem, and here is a mechanism to redeploy it toward the actual bottleneck.”

This reframing matters because it removes the ask from the category of “additional cost” and places it in the category of “corrected deployment of

existing commitment.” That is a different conversation, and it is one that the commercial side of the organisation can engage with on its own terms.

The discipline here is not financial cleverness. It is the same structural clarity applied to a different domain: understanding what the organisation’s actual constraints are, and designing an intervention that operates within them rather than demanding that they be relaxed.

5. The Cost of Duplication

One of the most common failure modes in scaling organisations is the duplication that emerges when platform investment is deferred and delivery teams fill the vacuum themselves. Each team builds its own implementation of shared concerns — regulatory reporting, reconciliation, account servicing — producing solutions that are locally coherent but globally incompatible.

This is not irrational behaviour on the part of the teams involved. They have delivery commitments and are being held accountable for them. Waiting for a platform that is currently a proposal feels riskier than building something imperfect but immediate. The incentive structure is pointing them toward duplication, not because they are wrong about their immediate situation, but because the organisation is failing to price in the long-term cost of divergence.

That cost is real and eventually becomes existential. Multiple incompatible implementations of regulatory reporting do not merely create technical debt. They create audit exposure, reconciliation failures, and compliance risk that no individual team has the authority or visibility to resolve. The problem has been distributed across the organisation in a way that makes it structurally invisible until it surfaces as a crisis.

There is also an ownership problem. When a delivery team co-opts responsibility for a domain it is not sufficiently familiar with — and which it should not, structurally, be responsible for — it creates a dependency between that team's continued participation and the organisation's ability to maintain the function. This is the guerrilla situation reproduced at organisational scale: a hostage taken not by individual initiative but by the accumulated effect of structural neglect.

Making this cost visible — in terms of regulatory exposure, operational risk, and the compounding overhead of maintaining divergent implementations — is part of the legitimate path. The argument cannot remain architectural. It must be translated into the language of risk and commercial liability that the organisation's decision-makers actually use.

6. The Boundary Condition

The thesis of this paper — that doing it right produces the right result — must be stated with honesty about its limits.

There are organisations that genuinely want to do everything right, have the people capable of doing it, and still fail. Not because the diagnosis was wrong or the disciplines were not applied, but because the accumulated structural debt had consumed so much of the operational margin that there was no runway left to correct it. The mess was a prior claim on the future that could not be discharged fast enough. These organisations go under. Everyone involved did what they could, and it was not enough.

This does not falsify the thesis. It reveals its boundary condition.

The thesis holds within a domain: organisations that have sufficient structural integrity to eventually receive a correct argument, and sufficient runway to act on it. Outside that domain — in organisations where the debt is terminal and the time has run out — no approach guarantees success. The legitimate path cannot rescue an organisation from consequences it has been accumulating for years. What it can do is ensure that those consequences are faced clearly, with the best available diagnosis, by people whose professional credibility and integrity are intact.

This boundary condition is itself an argument for starting earlier. Guerrilla development, accumulated technical debt, deferred structural investment, and the organisational entropy of duplication are not merely inefficiencies. They are, over time, existential risks. The terminal case is not a counterexample to the thesis. It is the thesis taken to its conclusion: this is what happens when the structural problems are not addressed through legitimate means, early enough, with sufficient seriousness.

There is a second limit worth naming. The legitimate path is also the mechanism by which an engineer correctly identifies when an organisation is incapable of receiving a correct argument — not merely slow, but structurally unable to act. In that case, doing it right means recognising the situation clearly, documenting the diagnosis, and making an informed decision about whether to remain. Leaving with credibility and a clear analysis intact is itself a form of the right thing happening. It is not failure.

7. The Promise

I said at the outset that the thesis is simple, and that I mean it without qualification: if you do this right, the right thing will happen.

I have now described what “right” means in sufficient detail to defend that claim. It means holding a correct position under grinding pressure, formally and visibly, without improvising outside the system of record. It means reading authority correctly and having the argument with the people who can act, not just the people who can agree. It means treating obstacles as stakeholders and designing for their requirements rather than around them. It means working within the resource logic of the organisation rather than demanding that it be suspended. And it means making the cost of inaction visible in terms the organisation’s decision-makers can receive.

None of this is easy. It is not meant to be. The legitimate path is harder than the guerrilla path in almost every dimension that matters to the engineer in the moment: it is slower, less satisfying, more exposed to the risk of being blocked, and offers no relief from the frustration that produced the temptation in the first place.

What it offers instead is something more durable. A solution that the organisation understands, sanctions, and can govern. A diagnosis that remains visible and traceable. A professional position that is credible regardless of whether the immediate outcome goes the right way. And, over time, in organisations capable of receiving it, the right result.

The engineer who does this right is not just solving a technical problem. They are creating the conditions under which technical problems of this kind can be solved — not once, as a *fait accompli*, but durably, as a function of how the organisation understands and governs itself. That is worth the grinding. It is worth the exhaustion. It is worth the patience. Do it right.